

wqf4w6d9c

March 11, 2026

1 Q-Learning en CliffWalking-v1

Estudio del algoritmo de Diferencia Temporal (TD) Off-Policy

Este notebook implementa y analiza el algoritmo **Q-Learning** sobre el entorno **CliffWalking-v1** de Gymnasium. A diferencia de los métodos de Monte Carlo, Q-Learning utiliza **bootstrapping**, permitiendo al agente aprender de sus estimaciones actuales sin esperar al final del episodio.

1.0.1 Q-Learning

- **Aprendizaje Off-Policy:** El agente aprende directamente la política óptima q_* , independientemente de la política de comportamiento usada para explorar. La acción del target siempre es el max sobre todas las acciones posibles, no la acción que realmente se tomará después.
- **Diferencia Temporal (TD):** Actualiza los valores Q basándose en la recompensa inmediata y el máximo valor estimado del siguiente estado:

$$Q(S_t, A_t) \leftarrow Q(S_t, A_t) + \alpha \left[R_{t+1} + \gamma \max_a Q(S_{t+1}, a) - Q(S_t, A_t) \right]$$

- **Comportamiento en CliffWalking-v1:** Las caídas durante la exploración sí generan actualizaciones negativas en la tabla Q , pero al ser off-policy el target usa $\max_a Q(s', a)$. Por eso la política greedy puede converger a la **ruta óptima** (13 pasos por el borde) aunque el comportamiento exploratorio siga siendo arriesgado. En SARSA (on-policy), la política aprendida tiende a ser más conservadora y alejada del acantilado.

1.1 1. Preparación del Entorno

La preparación consta de las siguientes partes: - **Instalación de dependencias:** librería `gymnasium`. - **Importación de librerías:** `numpy`, `matplotlib`, `tqdm`. - **Creación del entorno CliffWalking-v1:** espacio de observación `Discrete(48)` (grid 4×12), espacio de acciones `Discrete(4)` (arriba, derecha, abajo, izquierda), `render_mode='rgb_array'` para visualización como imagen. - **Importación de clases del repositorio:** `QLearningAgent`, funciones de visualización.

1.2 ##### Código de la Instalación e Importación

```
[1]: %%capture
    #@title Instalamos gymnasium
    !pip install gymnasium
```

```
[2]: #@title Importamos librerias
import sys
sys.path.insert(0, '../src')
import os
import numpy as np
%matplotlib inline
import matplotlib.pyplot as plt
from tqdm import tqdm
import gymnasium as gym

from agents import QLearningAgent
from plotting import plot, plot_lengths, print_q_summary, plot_comparison,
    plot_lengths_comparison, show_greedy_episode
```

```
[3]: # @title Semilla para reproducibilidad (sección 5.4 del PDF)
# Patrón recomendado: fijar la misma semilla en NumPy, Python y Gymnasium.
# Sin PyTorch porque este notebook es tabular (no usa redes neuronales).
SEED = 2024

# Fijar la semilla en NumPy
np.random.seed(SEED)
np.random.default_rng(SEED)

# Fijar la semilla en Python (evita variabilidad en hashing)
os.environ['PYTHONHASHSEED'] = str(SEED)

print(f"Semilla fija: SEED = {SEED}")
```

Semilla fija: SEED = 2024

```
[4]: #@title Importamos el entorno CliffWalking-v1
env = gym.make('CliffWalking-v1', render_mode='rgb_array')
env.reset(seed=SEED) # Fija la semilla del entorno (patrón sección 5.4 del
    PDF)

nS = env.observation_space.n # 48
nA = env.action_space.n # 4
print(f"Estados: {nS}, Acciones: {nA}")
print(f"Recompensas: -1 por paso | -100 caída al acantilado | 0 al llegar")
print()
```

Estados: 48, Acciones: 4

Recompensas: -1 por paso | -100 caída al acantilado | 0 al llegar

1.3 2. Diseño del Agente

El diseño del agente sigue el esquema de Gymnasium (sección 5.2): `__init__`, `get_action`, `update`, `stats`.

1.3.1 Política Epsilon-Greedy

- El agente utiliza una política ϵ -**soft** para garantizar la exploración continua del espacio de estados.
- Con probabilidad $1 - \epsilon$, selecciona la acción con el mayor valor Q (explotación).
- Con probabilidad ϵ , selecciona una acción al azar (exploración).

1.3.2 Algoritmo Q-Learning

- Implementado en la clase `QLearningAgent`, este algoritmo actualiza la tabla Q en cada paso del entorno.
- A diferencia de SARSA, Q-Learning es off-policy porque su target usa siempre $\max_a Q(S', a)$ independientemente de la acción que realmente tomará el agente. Recordemos:

$$Q(S_t, A_t) \leftarrow Q(S_t, A_t) + \alpha \left[R_{t+1} + \gamma \max_a Q(S_{t+1}, a) - Q(S_t, A_t) \right]$$

```
[5]: UP, RIGHT, DOWN, LEFT = 0,1,2,3
ACTION_NAMES = {0: 'N↑', 1: 'E→', 2: 'S↓', 3: 'W←', 4: 'PU', 5: 'DO'}
# La política greedy se obtiene con agent.pi_star(seed=SEED,
↪ action_names=ACTION_NAMES)
```

1.4 3. Experimentación

1.4.1 3.1 Representaciones Gráficas

Gráfica 1 — Recompensa media acumulada: $f(t) = \frac{\sum_{i=1}^t R_i}{t}$. En CliffWalking-v1 las recompensas son negativas (-1 por paso, -100 por caer al acantilado). Como esta métrica es acumulada, si el agente cae con frecuencia al acantilado durante muchos episodios, el promedio puede estabilizarse en valores muy por debajo de -13, aunque la política greedy final sea óptima.

Gráfica 2 — Longitud de los episodios: Mide cuántos pasos tarda el agente en llegar al objetivo o caer al acantilado. 1. **Inicio:** Episodios muy largos y frecuentes caídas al acantilado (penalización -100 y reset al inicio). 2. **Aprendizaje:** La longitud se estabiliza a medida que el agente aprende a navegar hacia el objetivo. 3. **Convergencia:** Las configuraciones que aprenden tienden a episodios cortos; en este experimento la estándar se estabiliza ~17 pasos, mientras que $\epsilon = 0.01$ se acerca a ~13 pasos (ruta óptima).

Función `show_greedy_episode`: Permite observar el comportamiento final del agente sin exploración, verificando si ha aprendido a cruzar el grid sin caer al acantilado y llegar al objetivo en el menor número de pasos posible.

1.4.2 3.2 Q-Learning en CliffWalking-v1

Se entrenan **10 000 episodios** con ϵ con decaimiento: $\epsilon = \min(1.0, 1000/(t + 1))$.

El decaimiento es necesario porque: - Al inicio, ε alto favorece exploración amplia del grid, incluyendo zonas cercanas al acantilado. - Al final, $\varepsilon \rightarrow 0$ hace que el agente explote la política aprendida, navegando directamente al objetivo.

Al ser off-policy, el decaimiento de ε solo afecta a la política de comportamiento (la exploración), no al target de aprendizaje. Las caídas durante la exploración sí generan actualizaciones negativas, pero el target usa la mejor acción estimada; por eso la política greedy puede converger a la ruta óptima a pesar de un comportamiento exploratorio arriesgado.

La semilla numpy fija (SEED = 2024) garantiza reproducibilidad.

```
[6]: # @title Aprendizaje off-policy (baseline)
agent_Q = QLearningAgent(env, epsilon=1, alpha=0.1, discount_factor=0.99,
    ↪decay=True)
agent_Q.train(num_episodes=10000)
list_stats_Q, list_lengths_Q = agent_Q.stats()
Q_Q = agent_Q.q_values
```

```
0%|          | 0/10000 [00:00<?, ?it/s] 10%|          | 1002/10000
[01:20<11:34, 12.96it/s]
success: -64713.5700, epsilon: 1.0000

24%|          | 2383/10000 [01:25<00:04, 1627.66it/s]
success: -33871.7265, epsilon: 0.5000

34%|          | 3444/10000 [01:25<00:02, 2456.11it/s]
success: -22644.1253, epsilon: 0.3333

45%|          | 4472/10000 [01:25<00:01, 3116.53it/s]
success: -17014.4283, epsilon: 0.2500

56%|          | 5592/10000 [01:26<00:01, 3540.83it/s]
success: -13629.3174, epsilon: 0.2000

68%|          | 6834/10000 [01:26<00:00, 3944.76it/s]
success: -11370.6742, epsilon: 0.1667

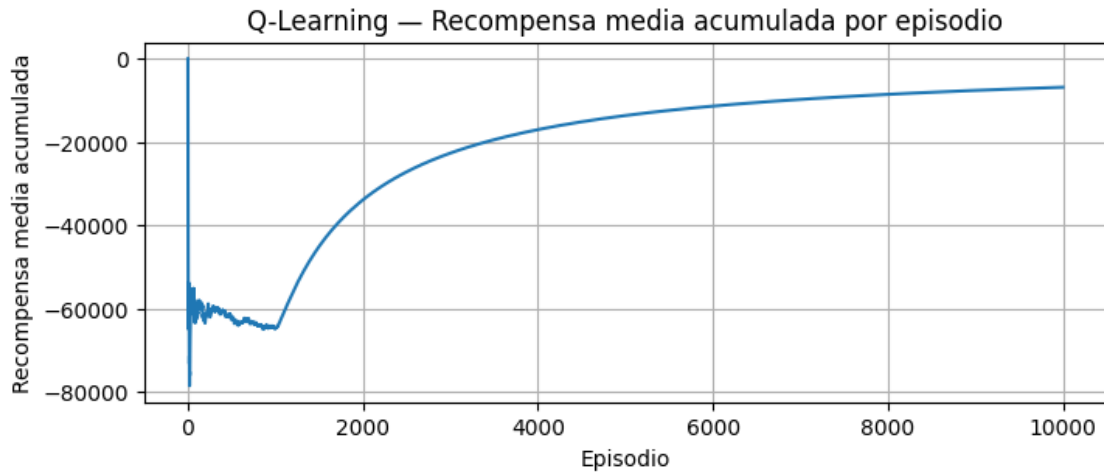
77%|          | 7687/10000 [01:26<00:00, 4114.15it/s]
success: -9755.4583, epsilon: 0.1429

86%|          | 8582/10000 [01:26<00:00, 4307.71it/s]
success: -8543.7229, epsilon: 0.1250

95%|          | 9481/10000 [01:27<00:00, 4410.22it/s]
success: -7600.0029, epsilon: 0.1111

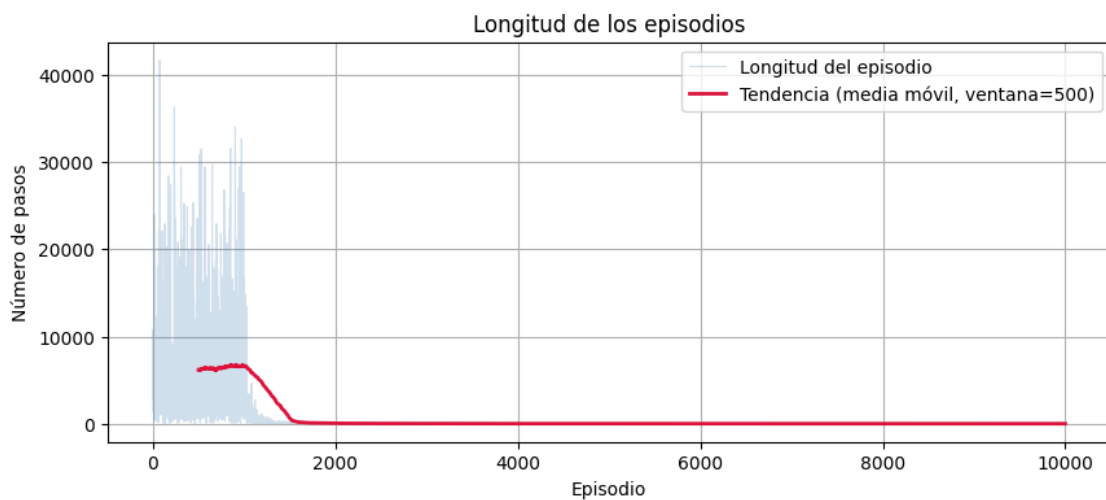
100%|         | 10000/10000 [01:27<00:00, 114.75it/s]
success: -6845.2253, epsilon: 0.1000
```

```
[7]: #@title Recompensa media acumulada por episodio (off-policy)
plot(list_stats_Q, title='Q-Learning - Recompensa media acumulada por_
↳ episodio', ylabel='Recompensa media acumulada')
print(f'Recompensa media final: {list_stats_Q[-1]:.2f}')
```



Recompensa media final: -6845.23

```
[8]: #@title Longitud de episodios (off-policy)
plot_lengths(list_lengths_Q)
print(f'Longitud media final (últimos 1000 episodios): {np.
↳ mean(list_lengths_Q[-1000:]):.2f} pasos')
```



Longitud media final (últimos 1000 episodios): 17.08 pasos

Resumen estadístico de la tabla Q aprendida. Con 48 estados y 4 acciones la tabla tiene 192 entradas. Se muestran estadísticas globales para verificar que el agente ha aprendido valores coherentes. En CliffWalking-v1 todos los valores Q son negativos o cero, ya que no existe ninguna recompensa positiva en el entorno — solo penalizaciones (-1 por paso, -100 por caída).

```
[9]: # @title Resumen tabla Q - off-policy
print_q_summary(env, Q_Q, title="Off-Policy - Tabla Q")
```

```
--- Off-Policy - Tabla Q ---
Entradas no nulas : 148/192 (77.1%)
Max Q              : 0.000
Min Q (no nulo)   : -112.125
Q media (no nulo) : -16.015
```

Se muestra la política greedy obtenida. `agent.pi_star(seed, action_names)` sigue $\arg\max Q[s]$ desde el estado inicial (esquina inferior izquierda, estado 36) y devuelve la secuencia de acciones en formato legible ($N \uparrow E \rightarrow S \downarrow W \leftarrow$). También se ejecuta `show_greedy_episode` para verificar visualmente la ruta aprendida.

```
[10]: # @title Política final (off-policy)
pi, actions = agent_Q.pi_star(seed=SEED, action_names=ACTION_NAMES)
print("Política greedy obtenida (off-policy)")
print(f"Acciones: {actions}")
print()
show_greedy_episode(env, Q_Q, seed=SEED, title="Q-learning - Episodio greedy")
```

Política greedy obtenida (off-policy)

Acciones: $N \uparrow \rightarrow E \rightarrow E \rightarrow E \rightarrow E \rightarrow E \rightarrow E \rightarrow E \rightarrow E \rightarrow E \rightarrow E \rightarrow E \rightarrow E \rightarrow S \downarrow$

Q-learning - Episodio greedy | Recompensa: -13 | Pasos: 13 | Terminado

Estado inicial:

```
[[[ 91 150  97]
   [ 91 150  97]
   [ 91 150  97]
   ...
   [ 86 170  69]
   [ 86 170  69]
   [ 86 170  69]]

[[[ 91 150  97]
   [ 91 150  97]
   [ 91 150  97]
   ...
   [ 86 170  69]
   [ 86 170  69]
   [ 86 170  69]]]
```

```

[[ 91 150 97]
 [ 91 150 97]
 [ 98 166 105]
...
[105 182 74]
[105 182 74]
 [ 86 170 69]]

...

[[ 86 170 69]
 [ 86 170 69]
 [ 93 188 74]
...
 [ 91 150 97]
 [ 91 150 97]
 [ 91 150 97]]

[[ 86 170 69]
 [ 86 170 69]
 [ 93 188 74]
...
 [ 91 150 97]
 [ 91 150 97]
 [ 91 150 97]]

[[ 86 170 69]
 [ 86 170 69]
 [ 86 170 69]
...
 [ 91 150 97]
 [ 91 150 97]
 [ 91 150 97]]]

Estado final:
[[[ 91 150 97]
 [ 91 150 97]
 [ 91 150 97]
...
 [ 86 170 69]
 [ 86 170 69]
 [ 86 170 69]]

[[ 91 150 97]
 [ 91 150 97]
 [ 91 150 97]
...
 [ 86 170 69]

```

```

[ 86 170 69]
[ 86 170 69]]

[[ 91 150 97]
 [ 91 150 97]
 [ 98 166 105]
...
[105 182 74]
[105 182 74]
[ 86 170 69]]

...

[[ 86 170 69]
 [ 86 170 69]
 [ 93 188 74]
...
 [ 91 150 97]
 [ 91 150 97]
 [ 91 150 97]]

[[ 86 170 69]
 [ 86 170 69]
 [ 93 188 74]
...
 [ 91 150 97]
 [ 91 150 97]
 [ 91 150 97]]

[[ 86 170 69]
 [ 86 170 69]
 [ 86 170 69]
...
 [ 91 150 97]
 [ 91 150 97]
 [ 91 150 97]]]

```

1.4.3 3.3 Modificación de parámetros de Q-learning

Los hiperparámetros en el algoritmo Q-Learning son fundamentales para equilibrar la explotación versus la exploración y cómo se gestionan las nuevas experiencias.

- La tasa de aprendizaje (α) controla qué tan rápido el agente actualiza sus valores Q ante nueva información; un valor elevado puede acelerar la convergencia inicial pero generar inestabilidad, mientras que un valor bajo asegura una convergencia más suave y robusta.
- El parámetro epsilon (ϵ) determina el grado de exploración del grid. En CliffWalking-v1, una exploración alta inicial expone al agente a frecuentes caídas al acantilado (-100), lo que deprime el promedio acumulado pero permite descubrir rutas. Al ser Q-Learning off-policy,

estas caídas no contaminan el target de aprendizaje.

- El factor de descuento (γ) dicta la importancia de las recompensas futuras frente a las inmediatas; un γ cercano a 1.0 es vital para que el agente planifique rutas largas hacia el objetivo, ya que todas las recompensas intermedias son -1 y solo el final del episodio termina la penalización acumulada.

```
[11]: # @title Aprendizaje off-policy
agent_Q = QLearningAgent(env, epsilon=0.01, alpha=0.1, discount_factor=0.99,
    ↪decay=False)
agent_Q.train(num_episodes=10000)
list_stats_Q_epsilon, list_lengths_Q_epsilon = agent_Q.stats()
Q_Q = agent_Q.q_values
```

```
16%|          | 1617/10000 [00:00<00:01, 4664.34it/s]
success: -25.9540, epsilon: 0.0100
success: -20.6330, epsilon: 0.0100

39%|          | 3931/10000 [00:00<00:01, 5579.25it/s]
success: -19.1330, epsilon: 0.0100
success: -18.3798, epsilon: 0.0100

57%|          | 5653/10000 [00:01<00:00, 5694.00it/s]
success: -17.9746, epsilon: 0.0100
success: -17.5890, epsilon: 0.0100

80%|          | 7951/10000 [00:01<00:00, 5603.02it/s]
success: -17.4913, epsilon: 0.0100
success: -17.4352, epsilon: 0.0100

100%|         | 10000/10000 [00:01<00:00, 5344.47it/s]
success: -17.3196, epsilon: 0.0100
success: -17.1807, epsilon: 0.0100
```

```
[12]: # @title Aprendizaje off-policy
agent_Q = QLearningAgent(env, epsilon=1, alpha=0.5, discount_factor=0.99,
    ↪decay=True)
agent_Q.train(num_episodes=10000)
list_stats_Q_alpha, list_lengths_Q_alpha = agent_Q.stats()
Q_Q = agent_Q.q_values
```

```
10%|          | 1001/10000 [01:21<09:47, 15.30it/s]
success: -65978.1170, epsilon: 1.0000

24%|          | 2396/10000 [01:26<00:04, 1613.00it/s]
success: -34832.5825, epsilon: 0.5000
```

```

35%|          | 3473/10000 [01:26<00:02, 2499.59it/s]
success: -23286.3363, epsilon: 0.3333
45%|          | 4464/10000 [01:27<00:01, 3057.93it/s]
success: -17494.9817, epsilon: 0.2500
56%|          | 5568/10000 [01:27<00:01, 3477.78it/s]
success: -14014.5268, epsilon: 0.2000
68%|          | 6808/10000 [01:27<00:00, 3928.46it/s]
success: -11691.3663, epsilon: 0.1667
77%|          | 7658/10000 [01:28<00:00, 4095.20it/s]
success: -10030.4721, epsilon: 0.1429
85%|          | 8504/10000 [01:28<00:00, 4158.66it/s]
success: -8784.1606, epsilon: 0.1250
99%|          | 9870/10000 [01:28<00:00, 4413.67it/s]
success: -7813.9047, epsilon: 0.1111
100%|         | 10000/10000 [01:28<00:00, 112.83it/s]
success: -7037.7481, epsilon: 0.1000

```

```

[13]: # @title Aprendizaje off-policy
agent_Q = QLearningAgent(env, epsilon=1, alpha=0.1, discount_factor=0.25,
    ↪decay=True)
agent_Q.train(num_episodes=10000)
list_stats_Q_discount, list_lengths_Q_discount = agent_Q.stats()
Q_Q = agent_Q.q_values

```

```

10%|          | 1002/10000 [01:19<10:32, 14.22it/s]
success: -64291.1370, epsilon: 1.0000
22%|          | 2199/10000 [01:24<00:05, 1428.79it/s]
success: -33960.7805, epsilon: 0.5000
35%|          | 3452/10000 [01:25<00:02, 2436.05it/s]
success: -22704.5167, epsilon: 0.3333
44%|          | 4449/10000 [01:25<00:01, 3041.41it/s]
success: -17060.0210, epsilon: 0.2500
56%|          | 5552/10000 [01:25<00:01, 3485.28it/s]
success: -13666.4056, epsilon: 0.2000

```

```

68%|          | 6785/10000 [01:26<00:00, 3908.44it/s]
success: -11401.2330, epsilon: 0.1667
76%|          | 7636/10000 [01:26<00:00, 4087.39it/s]
success: -9781.8186, epsilon: 0.1429
85%|          | 8509/10000 [01:26<00:00, 4226.50it/s]
success: -8566.3175, epsilon: 0.1250
99%|          | 9882/10000 [01:26<00:00, 4486.03it/s]
success: -7620.4901, epsilon: 0.1111
100%|         | 10000/10000 [01:26<00:00, 115.11it/s]
success: -6863.0571, epsilon: 0.1000

```

```

[14]: # @title Aprendizaje off-policy
agent_Q = QLearningAgent(env, epsilon=1, alpha=0.1, discount_factor=0.05,
    ↪decay=True)
agent_Q.train(num_episodes=10000)
list_stats_Q_bigdiscount, list_lengths_Q_bigdiscount = agent_Q.stats()
Q_Q = agent_Q.q_values

```

```

10%|          | 1001/10000 [01:20<09:31, 15.73it/s]
success: -65234.7330, epsilon: 1.0000
21%|          | 2093/10000 [01:26<00:09, 796.38it/s]
success: -34316.6760, epsilon: 0.5000
31%|          | 3105/10000 [01:27<00:07, 980.41it/s]
success: -22929.3723, epsilon: 0.3333
41%|          | 4087/10000 [01:28<00:06, 892.53it/s]
success: -17225.2035, epsilon: 0.2500
51%|          | 5076/10000 [01:29<00:06, 722.57it/s]
success: -13804.1326, epsilon: 0.2000
61%|          | 6118/10000 [01:31<00:06, 612.74it/s]
success: -11525.9730, epsilon: 0.1667
71%|          | 7103/10000 [01:32<00:04, 598.83it/s]
success: -9898.8894, epsilon: 0.1429
81%|          | 8081/10000 [01:34<00:03, 522.49it/s]
success: -8680.7624, epsilon: 0.1250

```

```

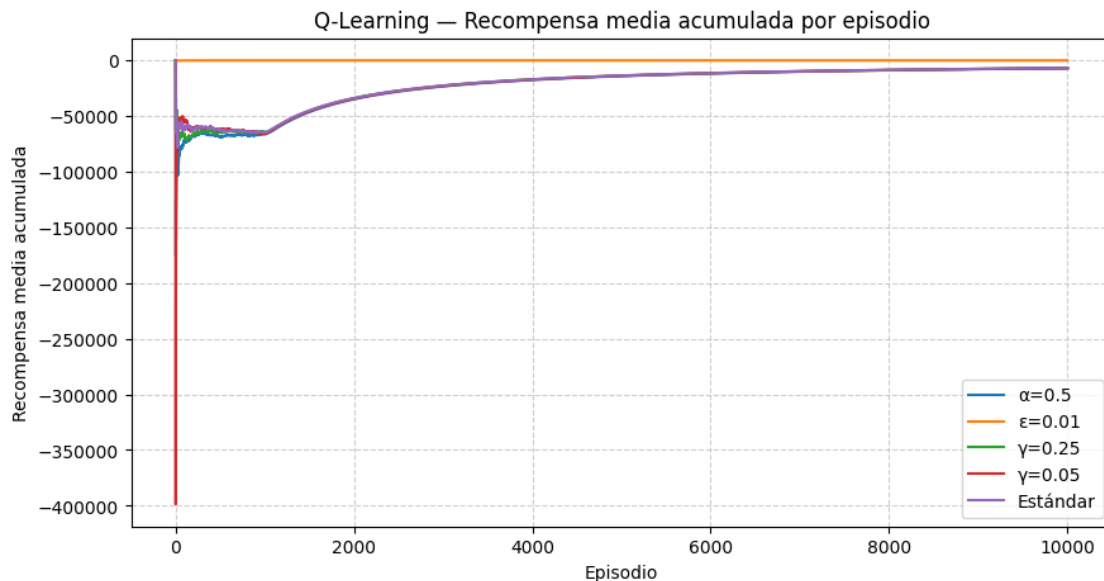
91%|      | 9098/10000 [01:36<00:01, 470.92it/s]
success: -7735.4584, epsilon: 0.1111
100%|     | 10000/10000 [01:39<00:00, 100.93it/s]
success: -6980.9861, epsilon: 0.1000

```

```

[15]: #@title Recompensa media acumulada por episodio (off-policy)
lista_plots_recompensa = [list_stats_Q_alpha, list_stats_Q_epsilon,
    ↳ list_stats_Q_discount, list_stats_Q_bigdiscount, list_stats_Q]
labels = ['=0.5', '=0.01', '=0.25', '=0.05', 'Estándar']
plot_comparison(lista_plots_recompensa, title='Q-Learning - Recompensa media
    ↳ acumulada por episodio', ylabel='Recompensa media acumulada', labels=labels)
for lista, label in zip(lista_plots_recompensa, labels):
    print(f'Recompensa media final de {label}: {lista[-1]:.2f}')

```



```

Recompensa media final de =0.5: -7037.75
Recompensa media final de =0.01: -17.18
Recompensa media final de =0.25: -6863.06
Recompensa media final de =0.05: -6980.99
Recompensa media final de Estándar: -6845.23

```

```

[16]: #@title Longitud de episodios (off-policy)
lista_plots_longitud = [list_lengths_Q_alpha, list_lengths_Q_epsilon,
    ↳ list_lengths_Q_discount, list_lengths_Q_bigdiscount, list_lengths_Q]
plot_lengths_comparison(lista_plots_longitud, labels=labels)
for lista, label in zip(lista_plots_longitud, labels):

```

```
print(f'Longitud media final (últimos 1000 episodios) de {label}: {np.
↪mean(lista[-1000:]).2f} pasos')
```



Longitud media final (últimos 1000 episodios) de $\alpha=0.5$: 17.19 pasos
 Longitud media final (últimos 1000 episodios) de $\epsilon=0.01$: 13.36 pasos
 Longitud media final (últimos 1000 episodios) de $\gamma=0.25$: 16.66 pasos
 Longitud media final (últimos 1000 episodios) de $\gamma=0.05$: 184.70 pasos
 Longitud media final (últimos 1000 episodios) de Estándar: 17.08 pasos

1.4.4 3.3.1 Análisis de los hiperparámetros

1.4.5 3.3.1 Análisis de Resultados y Comparación de Configuraciones

En esta sección se evalúa el impacto de los hiperparámetros en el aprendizaje de Q-Learning en CliffWalking-v1. Se han definido cuatro configuraciones específicas (más la original):

1. **Configuración 1** ($\epsilon = 0.01$) sin decaimiento: Minimiza la exploración desde el inicio. En CliffWalking-v1 esto es especialmente efectivo porque el agente evita las caídas al acantilado desde el principio, aprendiendo rápidamente una política útil.
2. **Configuración 2** ($\alpha = 0.5$): Dobra la tasa de aprendizaje, permitiendo actualizaciones más rápidas. En este experimento no muestra una mejora claramente superior frente a la configuración estándar.
3. **Configuración 3** ($\gamma = 0.25$): Reduce el descuento de recompensas futuras. En este experimento sigue permitiendo aprender una política funcional, pero no muestra ventaja clara frente a la configuración estándar y queda por detrás de $\epsilon = 0.01$.
4. **Configuración 4** ($\gamma = 0.05$): Descuento extremo. El aprendizaje es deficiente: episodios muy largos y recompensa media muy negativa.

Interpretación de las Gráficas Recompensa Media Acumulada: - La configuración estándar (con decaimiento desde $\epsilon = 1$) muestra un promedio muy negativo dominado por las frecuentes caídas al acantilado durante los primeros episodios de exploración aleatoria. - $\epsilon = 0.01$ es claramente la mejor configuración: al evitar la exploración masiva inicial, el promedio acumulado no se ve arrastrado por cientos de episodios con penalización de -100. - $\gamma = 0.25$ no muestra mejora frente a la estándar, y $\gamma = 0.05$ tiene un rendimiento claramente deficiente.

Longitud de los Episodios: - Las configuraciones que aprenden una política útil convergen a episodios relativamente cortos, mientras que $\gamma = 0.05$ no lo hace. - Las diferencias en longitud final entre las configuraciones que sí aprenden no son enormes, pero $\epsilon = 0.01$ es la que más se aproxima a la ruta óptima de 13 pasos.

1.4.6 Guardar métricas (Q-Learning)

Esta celda guarda las series para poder comparar con otros notebooks.

```
[17]: import os, numpy as np
os.makedirs('metrics', exist_ok=True)
np.savez('metrics/q_cliff.npz', list_stats=list_stats_Q,
        ↪list_lengths=list_lengths_Q)
print('Guardado: metrics/q_cliff.npz')
```

Guardado: metrics/q_cliff.npz

1.4.7 Comentario de la comparativa Q-Learning vs SARSA

- La **recompensa media acumulada** es muy negativa al inicio porque la métrica arrastra muchas caídas (-100) en las primeras fases de exploración.
- A medida que avanza el entrenamiento, **Q-Learning queda ligeramente por encima (menos negativo)** que SARSA, indicando mejor desempeño promedio acumulado.
- La separación entre curvas es moderada: ambos mejoran, pero el off-policy de Q-Learning se beneficia de un target greedy.
- Esta gráfica es acumulada; para validar la ruta óptima conviene mirar también el **episodio greedy final** (13 pasos, recompensa -13).

```
[6]: from pathlib import Path
import numpy as np

def _load_metrics(path):
    data = np.load(path, allow_pickle=True)
    return data['list_stats'], data['list_lengths']

q_path = Path("metrics/q_cliff.npz")
s_path = Path("metrics/sarsa_cliff.npz")

if not (q_path.exists() and s_path.exists()):
    raise FileNotFoundError("Faltan metrics/q_cliff.npz o metrics/sarsa_cliff.
    ↪npz")
```

```

list_stats_Q, list_lengths_Q = _load_metrics(q_path)
list_stats_S, list_lengths_S = _load_metrics(s_path)

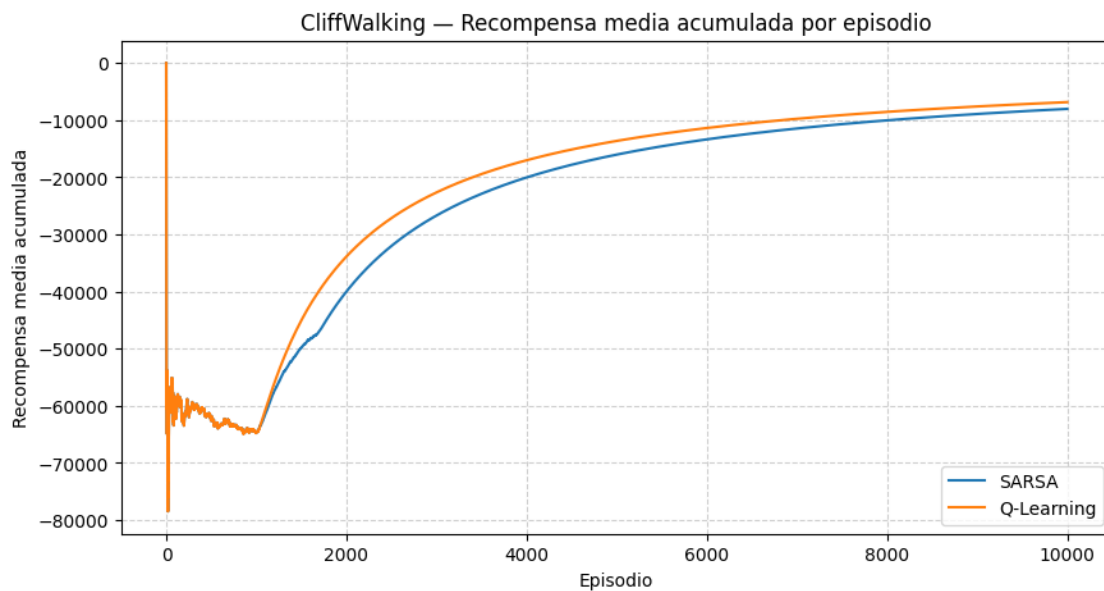
LABELS = ["SARSA", "Q-Learning"]

plot_comparison(
    [list_stats_S, list_stats_Q],
    labels=LABELS,
    title="CliffWalking - Recompensa media acumulada por episodio",
    ylabel="Recompensa media acumulada"
)

plot_lengths_comparison(
    [list_lengths_S, list_lengths_Q],
    labels=LABELS
)

for stats, label in zip([list_stats_S, list_stats_Q], LABELS):
    print(f"{label:10s} - Recompensa media final: {stats[-1]:.2f}")
for lengths, label in zip([list_lengths_S, list_lengths_Q], LABELS):
    print(f"{label:10s} - Longitud media final (últimos 1000 ep.): {np.
    ↳mean(lengths[-1000:]):.2f} pasos")

```





SARSA - Recompensa media final: -8025.64
 Q-Learning - Recompensa media final: -6845.23
 SARSA - Longitud media final (últimos 1000 ep.): 19.33 pasos
 Q-Learning - Longitud media final (últimos 1000 ep.): 17.08 pasos

1.5 4. Análisis y Conclusiones

1.5.1 4.1 Comparativa de Desempeño frente a Monte Carlo

Q-Learning converge a una política compatible con la óptima en CliffWalking-v1 de forma notablemente más rápida que los métodos de Monte Carlo. Al actualizar Q en cada paso, el agente aprende desde los primeros episodios a evitar el acantilado (penalización -100), sin necesidad de esperar al final del episodio como requiere MC.

La longitud media final (últimos 1000 episodios) es de **~17 pasos** en la configuración estándar, mientras que la política greedy final sí sigue la trayectoria óptima de **13 pasos**: $N \uparrow \rightarrow E \rightarrow \times 11 \rightarrow S \downarrow$. Esto es consistente con Q-Learning off-policy: puede aprender la ruta óptima aunque la exploración ϵ -greedy mantenga episodios algo más largos en promedio.

La tabla Q muestra valores mayoritariamente negativos (no hay recompensas positivas). Las estadísticas concretas de la tabla deben leerse desde la salida del resumen (`print_q_summary`) del run actual, ya que varían con la semilla y el régimen de exploración.

Al observar un episodio greedy, el agente sigue la ruta óptima sin desviarse, confirmando que ha aprendido correctamente la dinámica del entorno: el camino más corto posible, aún sabiendo que pasa a un paso del acantilado.

1.5.2 4.2 ¿Por qué Q-Learning funciona mejor que Monte Carlo en este entorno?

Q-Learning supera a Monte Carlo principalmente en velocidad de convergencia. La razón es el **bootstrapping**: en lugar de esperar el retorno acumulado del episodio completo, Q-Learning actualiza en cada paso usando la estimación del valor del siguiente estado. Esto es especialmente ventajoso en CliffWalking, donde los primeros episodios con ε alto generan caídas frecuentes al acantilado: MC propaga esa penalización de -100 solo al terminar el episodio, mientras Q-Learning la incorpora inmediatamente en el paso en que ocurre.

Además, al ser off-policy, Q-Learning aprende el valor de la acción greedy aunque esté ejecutando ε -greedy. Esto permite que la política aprendida converja hacia la óptima (ruta por el borde del acantilado) incluso durante la exploración, algo que SARSA no puede hacer porque su actualización incluye la acción exploratoria real.

1.5.3 4.3 Propuestas Futuras

- Comparar directamente con **SARSA** para observar cómo el carácter on-policy lleva a SARSA a preferir la ruta segura (17 pasos por la fila superior) frente a la óptima de Q-Learning (13 pasos por el borde del acantilado). Esta diferencia es el Ejemplo 6.6 clásico de Sutton & Barto (2018).
- Evaluar el impacto del tamaño del grid (mapas más grandes) sobre la ventaja de TD frente a MC en términos de episodios necesarios para converger.